ספרהוב · Seferehov — features, hardware and capacity

Updated 2026-10-21

- What it is
- Features
  - Map and street libraries
  - Adding books — three ways
  - Reading the photos (on the server)
  - The approval queue
  - Reading order (when many readers scan at once)
  - My books (home library)
  - Books for sale or to give away
  - Finding books
  - Reviews, fixes, points
  - App and updates
  - For the admin
  - Measured accuracy (2026-10-21)
  - Tests (from 11 Nov 2026)
- Hardware
- Capacity: how many scans the GPU can handle
  - Ways to grow, in order
- Disk space
- Backups
- Future plans
- Keeping this document current

What it is

Seferehov (ספרהוב, Hebrew for “street books”) is a web app, installable on any phone, where books pass from hand to hand. It has three parts:

- a shared map of free street libraries (book boxes, little free libraries, shelves in lobbies and parks) and the books on their shelves now;
- readers’ own home libraries, catalogued from photos;
- books readers offer from home, for sale or to give away.

It runs at https://seferehov.com. Hebrew is the default (right to left); English is one tap away. All the AI runs locally on one home server, so there is no per-use AI cost and no photos are sent to an AI company.

Features

Map and street libraries

- **Map** of about 48,000 street libraries worldwide, seeded from OpenStreetMap (`amenity=public_bookcase`). About 70 of them are in Israel. When zoomed out, libraries are grouped into number bubbles.
- **Library page**: photo, hint for finding it, the books seen there lately, books taken or gone in the last 90 days, and spines read from photos that no catalogue knew yet (so the next visitor can photograph their covers).

- **Add a library**: GPS position, a photo and a hint. A double tap does not create two libraries.
- **Suggest changes**: new name, hint, photo, position, or “it’s gone”. New libraries and changes wait for a reviewer, and admins get an e-mail digest.
- **Duplicate libraries** within 20 m are listed for merging; old links redirect to the kept one.
- **OpenStreetMap twins merged by themselves**: when a reader scans books at a library they added and an OpenStreetMap library is within 40 m, the OpenStreetMap entry is merged into the reader’s (its name and description fill gaps; its link redirects; a new OpenStreetMap import doesn’t bring it back). Not when the OpenStreetMap entry has books, scans, checks or edits of its own, or a reader marked the two as different: those stay on the duplicates page.
- **Printable sign** (A5 with a QR code) for each library.
- **Private-info check**: names, hints and notes are checked for door codes, phone numbers and e-mails (patterns plus the local AI model) before they are published.
- **Town pages and country pages** for search engines, with the libraries and books in each town (towns come from GeoNames and OpenStreetMap boundaries).
- Only libraries with something of their own (added, checked, edited or reported by readers, or in Israel) are in the sitemap and open to search engines; the untouched OpenStreetMap shelves abroad, and towns and countries made only of them, get `noindex` so Google doesn’t treat them as duplicates.
- **Asking for cover photos**: a library’s page tells how many of its books have no photo of their front and back covers yet, and invites a reader who is there to photograph them with the main camera (📸 Photograph books, a button right there): no searching for each book’s page. Several fronts and then their backs in the same order is fine. The server works out which book each is (the photo sorting pairs the k-th front with the k-th back as well as covers taken one after the other; on 6 real books photographed as 6 fronts then 6 backs it paired 4 right, against 1 before), and covers whose title matches a spine read at that library that no catalogue knew are folded into it, so the spine’s finder gets points too. The phone says “Thank you!” when the photos are sent. Each such book is marked “📷 needs cover photos”. A book needs them when it has no picture of its front at all, or no back cover (SQL functions `book_lacks_front` / `book_lacks_back`).
- **Points for missing covers**: the first photo of a book’s front earns 30 points and of its back 40 (on top of the book on the shelf), once approved.
- **Where you can help** (on the 🏆 page): up to 10 libraries with books lacking cover photos or spines waiting for covers, nearest first when the phone already allows its position (it never asks for it there), with the points on offer. From the daily job, not worked out per visit.
- **Not seen lately**: a book on a library’s list that readers’ shelf photos on later days didn’t show is marked “not seen in the last shelf photos: maybe taken, maybe just on a shelf nobody photographed”. It is never marked gone by itself: a reader rarely photographs every shelf.

Adding books — three ways

1. **One camera for everything, free style** (main way, at a library and at home, `web/src/capture.js`): one button, “Photograph books”. The reader points at a shelf, a front cover or a back cover, in any order, and stops for a moment; every stop is a photo. The phone

does not decide what a photo is: the server does. (Until 4 Nov 2026 the covers camera decided on the phone — turned over = the same book, moved sideways = the next book — and on a real session of books standing on a shelf it split 3 books into 6: a book turned over on a shelf moves in the picture just as the phone moving on does.)

- **When a photo is taken.** About 7 times a second the phone lines up a tiny grey copy of the picture with the one before, which gives the phone's real speed and how far it has moved since the last photo. Once the picture is new (moved on by 30% of it, or changed a lot: a book turned over, the next cover) and the phone is held there — steady for 0.45 s, roughly steady for 0.9 s, or, for a trembling hand, at the latest after 1.6 s — the best frame of the pause is kept at full camera resolution (`web/src/steady.js`). "Held there" means the picture's net drift over the last second is small, however fast it trembles: only drift or a big jump means moving on. Each reader's own tremor is learnt and kept on the phone, and the steady and rough limits grow with it. The best frame is the sharpest one where the picture and the phone's gyroscope were calmest. A picture lost for one check while the gyroscope says the phone was calm (or, without a gyroscope, for a single check) is a glitch, not a move. The full-size photo is encoded to JPEG off the page's thread (`web/src/encode.js`, a worker; the canvas's own encoder where there is none): on a reader's Android 10 phone each 4K photo took about 9 s to encode, and the checks stalled meanwhile to one every 1.4 s, half of them "lost". (Before 12 Nov 2026 the phone had to be steady or roughly steady; readers' sessions showed the "slower" sign 40% of the time and photos 2–6 s after stopping, and a simulated trembling hand got no photo at all in 12 s; now it gets one at every stop.) If the phone is still held there 0.35 s later, a second sharp frame is taken quietly (`meta.extra`): the server pools the readings of every photo a spine is in and shows the best-cut picture of it. (Before 6 Nov 2026: 40%, 0.6 s, 1.6 s, one photo per pause.) A bar shows "Hold steady…" filling up, then "✓ Taken — turn the book over or move on".
- **What goes with each photo.** Its number in the session, the time since the camera opened, a barcode the live camera saw just before (the browser's barcode reader looks every 0.6 s while the phone is still), the picture's travel since the last photo and the phone's own movement from its motion sensors (summed acceleration, turning). The movement is logged for later; the pairing does not use it yet. Each photo also carries how it was taken (`steady`, `rough`, `shaky`, `extra`), the reader's learnt tremor and the gyroscope's speed at that moment; the session's report gets the tremor and the time between checks (event `hand`).
- **Sent one by one.** Each photo is saved on the phone and sent by itself at once (store and forward, below), to `POST /api/captures`. When the camera closes for any reason the server is told how many photos were taken (`POST /api/captures/<session>/end`).
- **Stopping.** A big "■ Done, send · " button sits on the camera picture itself (it used to be under the photos, off the screen on a phone), plus ✕ in the corner. After stopping, the panel follows the uploads — "5 of 12 photos sent…", then "All 12 photos are on the server. You can put the phone away." — so the reader knows when it is safe to switch the screen off. Switching the screen off with the camera open still works (every photo is already saved on the phone), but the phone may hold the uploads until the app is opened again.
- **Sorted on the server** (`server/src/capture.js`, `server/src/pairing.js`) as soon as all the photos are there; if the phone never said "end" (the page died), 3 minutes after the last photo; if photos are still missing, 30 minutes after. For each photo the vision model says shelf, front, back or other (a second photo of a pause takes the kind of the one before, and a cover's second photo joins its group), and barcodes are read (a cover with a barcode is a back). Covers are cut out; the same side photographed twice is grouped

(grey-picture likeness, colours, and the model when unsure). Fronts and backs no more than 5 photos apart are scored: colour likeness (a book's two covers share paper, colours and publisher's logo), closeness in the session, a back after its front, and the vision model shown both covers ("the same book?" — it says yes too easily, so it only tips close calls). Best pairs first; leftovers are books with one side only. Runs of shelf photos (up to 8 places; second photos of a pause not counted) become a shelf scan, each book a covers scan, read as before; everything waits in the approval queue. A book with only its back cover is read from the back (barcode, and the title its blurb names), marked low quality. Photos arriving after their session was sorted are sorted on their own. The sorting is traced (`/admin/trace`, `capture-…`).
    - **Tested** on the real session that went wrong (10 photos: 3 books front and back, fronts photographed twice, one back alone): sorted and paired 10/10 right, about 4 minutes on the home server.
2. **Photos from the gallery**: up to 40 photos at a time, shelves or covers, sorted and paired the same way (only their order is known).
3. **Barcode scanning** (library page): the camera keeps scanning, each ISBN goes to the queue.

One book's covers on their own (the covers camera, `web/src/shots.js`): used where a single book is asked for — a missing side on a book page, a spine found in no catalogue (library check), an unknown barcode. The live camera takes each picture by itself when the book is held steady (it accepts more shake the longer it waits, so a photo comes within 1–2 seconds), shows the book turning over between sides, and looks for the barcode the whole time. If none was seen, it asks for a 4-second close-up of the barcode. Then, for 6 seconds, the book's two pictures are shown over the camera with "retake" on each (the first take is sometimes too quick; tapping one goes back to that side) and a bar running down; untouched, or once the next book's front is taken, the book is sent by itself. No "keep" button; a wrong book is declined in the approval queue. - **The whole book must be in the picture.** A small model on the phone (a layer on the same image embedder as the book detector, `web/src/wholehead.json`) says whether all four corners are in view; if not, the photo is held back and the camera says to move back a little, because the server cannot cut out and straighten a cover that is partly off the picture. It was trained on real cover photos of this app, cut so that the cover fits or does not, and on Open Library covers on real surfaces. On real photos it had not seen: 91% of whole covers accepted, 87% of cut ones caught (90% when a quarter or more is missing). After 6 seconds of holding back the photo is taken anyway, so a cover the model misjudges cannot stop the camera; the camera report counts these. A phone that cannot run the model takes photos as before. - **More frames.** Right after each cover photo, while the phone is almost not moving, up to two more frames (1600 px) go with it. No extra time for the reader. For Hebrew covers the server reads each with OCR: lines that are the same text vote, and every frame's spelling of the title is also tried against the catalogue; a frame after the back cover is also searched for the barcode. Tested on a real shelf video: frames differ in which words they get right, while one sharpened picture stacked from the frames read far worse than any single frame, so frames are read, not merged. The shelf camera still sends one frame per pause (its reading takes minutes per photo).

Store and forward (nothing taken is lost, no button to press). Every photo, cover, shelf batch, barcode and new library is written to the phone first (IndexedDB) and sent from there by an outbox (`web/src/queue.js`):

- A photo is saved the moment it is taken, into a "held" batch (the shelf camera's photos, a book's two covers) that is sent once complete or as soon as the camera closes for any reason. A batch left by a page that is gone (closed, reloaded, crashed) is sent by itself after 3 minutes.

- No signal, a timeout, the server busy or restarting (5xx, 429): kept and tried again with growing pauses (15 s up to 15 min), at once when the phone is back online or the app is opened. Signed out (401): kept until sign-in. App too old (426): kept and sent by the updated app (updates no longer wait for the outbox to be empty, only for a send in progress or a batch being taken).
- The service worker sends too (Background Sync): uploads left when the app is closed go when the phone has signal (Chrome/Android).
- Refused by the server (another 4xx): kept as “not sent” with the reason; the header badge says so and lists them, to try again or delete; also retried when the app starts (up to 6 times). A refusal the reader is watching (a mistyped ISBN) is said at once instead.
- A barcode the catalogue knows goes straight to the library (or My books); one it does not know falls back to sending the covers to be read.
- The badge lists what is still on the phone, each with its picture, when it was taken, what went wrong the first time and the latest time, in plain words, and how many tries. “Try again” sends them at once, and the list comes back saying what that try did (how many went, how many are still there).
- A barcode is credited only to the book in view: one read while the picture has not moved on since the last photo belongs to that last book, and a barcode that already went with a book in this session is never given to another.
- A barcode no catalogue knows (usual for a Danacode, the code on most Israeli books, the first time anyone scans that book): the covers go to be read instead, with the barcode, and the book waits in the approval queue.
- Every upload carries an id (`X-Upload-Id`); the server keeps the answer 14 days (`upload_receipts`, `server/src/receipts.js`), so an upload sent again because its answer was lost is never saved twice.

Camera and upload reports (for later analysis, to improve scanning; no pictures, numbers only, kept a year): - Every live-camera session — free-style capture, one book’s covers, barcode scanning — sends a report when it ends, also when the camera would not open (`web/src/camlog.js` → `POST /api/camera-log` → table `camera_sessions`). The report goes before anything else is awaited (a page hidden by the screen going off may be frozen at any moment: a covers session’s report was lost this way on 3 Nov 2026), is trimmed to fit the 64 KB a beacon may carry, and is kept on the phone and sent with the next session if it could not go. - The phone: model, system and version (Android Chrome gives them directly; iPhones through the browser’s user-agent text), browser and full version, screen size and pixel ratio, CPU cores, memory, network type and speed, storage used, graphics chip, installed app or browser tab, which camera features the browser has. - The camera actually given: resolution, frame rate, focus, exposure, zoom, torch, what it can do at most, and the largest real photo it can take. - Timings: how long the camera took to open and show a first picture; each photo’s time, how it was taken (shelf: steady / rough / leaving; covers: still / sharpest-of-a-moment / button / camera app), the wait from “ready” to the photo, sharpness, shake, glare or dark, size, time to encode it; each book’s time from front to back and in all, retakes, how its barcode was found (live camera, from the photo, or none) and when; time spent in each camera state; barcode-detector type and time per look; the book detector’s time per look (many books). - Each upload from the outbox (`upload_timings`, `server/src/receipts.js`): how long it waited on the phone (no signal, batch still being taken), how long the upload took on the phone, how long the server took, try number, size, network, page or service worker, and what it became (scan, book or sighting id). The camera session id ties uploads (and so scans and their traces) to the session. - Each line on screen follows its upload: saved on the phone, sending, sent (with the book’s title when known), waiting (no signal / sign in / server busy), or not sent.

Reading the photos (on the server)

- **Shelf photos**: the local vision model (Qwen3-VL 8B) finds each spine, straightens the photo by the spines' lean, cuts out each spine, and reads it. A second reader (Surya OCR) reads the same cut-out. A reading is kept when the two agree. Readings of the same spine in different photos are merged by similarity, and the most frequent spelling wins.
    - Shelf photos arrive at the camera's full size (2160×3840 from the live camera, JPEG encoded once); a spine is only 100–200 px wide, so every pixel counts.
    - A blurred photo (sharpness — variance of the Laplacian at 1200 px — under 250; measured: under ~150 nothing was read, from ~475 most spines were) is skipped when the scan has a sharp photo.
    - Which way up: the way where the model and Surya agree; when Surya's lines match both ways (it reads upside-down print too), the way the model read something wins.
    - The final cut is made at each spine's own angle (`spineCut`, step "Cutting the spine at its own angle"): the lean of the spine's own print is measured, and the cut is tried both as it is and turned level for that lean. Above and below the print, the spine's edge is searched for as a straight line at any angle up to ±6° (the strongest colour change along a line that does not cross the print; a line may reach a little into Surya's padded text box, and further only when it is a much clearer edge than any other). The version with the stronger, more level edges is kept; if the two edges disagree by more than 3°, the clearer one is kept and the other side is cut level. Whatever lies beyond the edges (a neighbour leaning another way) is painted light grey. With no clear edge: the level cut, as before. Two dark spines with no visible edge between them still come out together.
    - When looking for a title the whole-photo read found, the cut-out's title must be confirmed by Surya, not only its author (the model sometimes takes the title from one spine and the author from the next).
    - A box holding two spines (the model read one, Surya the other, so they never agree): cut into one band per spine along Surya's lines, each band read on its own. Alike readings from overlapping cut-outs count once.
    - A book only the whole-photo read found is located by its title and cut out; if that spine is one already read (same place, alike reading), the whole-photo reading is dropped instead of queued twice under a misread title; otherwise the cut-out's own reading replaces the whole-photo one.
- **Covers**: the cover is cut out and straightened, then read. Hebrew text is read by Surya OCR and corrected with the hspell dictionary; Latin text by the vision model. Hebrew the model writes back to front is turned round.
- **Hebrew covers, title and author**: the vision model understands the layout (which text is the title, which the author) but invents Hebrew letters it cannot make out; Surya OCR copies the letters exactly but sees only lines. So both read the front, and the model also picks the title and author among the OCR lines. Each field the model read is replaced by the OCR line it resembles; when the two spellings differ, the one with more words the dictionary knows wins. A reading with the author first is turned round.
- **Local title index** (`server/src/refindex.js`, step "Local title index"; filled by `server/scripts/import-refs.js`, refreshed monthly): titles from the bibliography sections and book infoboxes of Hebrew Wikipedia, Hebrew-titled written works from Wikidata, and author names from Wikidata (people in the National Library's name file, authors of works, writers and poets), Hebrew Wikipedia and Project Ben-Yehuda. Matching uses a folded key where look-alike letters (ז/ו, ח/ת/ה, כ/ב, ד/ר), final letters and the freely written ו/י become one, then scores the real spelling: a candidate is taken when its title is very close to the reading (or to a part of

it) and the author on the spine is not someone else's, or when nearly all its words and its author are close to what the spine says; titles under 8 letters always need the author. When only the author is found, its spelling is corrected. A match here skips the web search. First import (2026-11-05): 217,803 titles (174,773 from Hebrew Wikipedia, 43,030 from Wikidata) and about 300,000 author names, 112 MB; the Wikipedia part takes 3 minutes to rebuild, Wikidata a few minutes more. Of the 124 Hebrew books in the catalogue at the time, 41% had their exact title in the index and 50% a close one; of those with a Hebrew author name, 61% had the author. On the 2026-11-04 shelf photo it corrected 3 of 6 misread spines by itself (אדונית התבלינים, אחוזות החוף, אוטופיה בכחול לבן) with no wrong matches.

- **Publishers' own websites in the title index** (`server/scripts/sites.js`, table `ref_sites`, daily timer `seferehov-sites`): Israeli publishers with a website are listed from Wikidata (296; 265 after dropping social networks and universities). Before anything is read, each site's robots.txt (for `SeferehovBot` and `*`, with Crawl-delay) and its terms of use are checked: every terms page the home page links to (shops often have separate site terms and seller or author terms), the strictest verdict winning. Terms that forbid robots make the site `refused`; terms that forbid copying make it `ask` (a person may ask the publisher), and the ban is read with the lines just before it, because Hebrew terms often say "חל איסור:" above a list. A large site (over 500 books) whose terms page could not be found, and a publisher's second domain when the first refuses, also wait for a person. Allowed sites are read through the shop's own product listing (WooCommerce Store API, Shopify), else their sitemap's book pages (schema.org Book/Product as JSON-LD or microdata, every book on a listing page), keeping only title, author, ISBN and page URL; sets, offers and shop notes ("אזל מהמלאי", "פגמים קלים בכריכה") are dropped. One request at a time, at least 12 s apart per site, at most 2,000 pages a day, a site paused at the first 403/429 or a Crawl-delay over 2 minutes, each site re-read every 90 days. The bot explains itself and how to opt out at `/bot`; `sites.js optout <domain>` removes a site's books. First check (2026-11-05): 132 sites allowed (57 with a book list, about 9,100 books by their shop or sitemap counts), 66 waiting for a person (about 62,000 books, mostly Indiebook, Bookpod, Ben-Zvi Institute and Keter), 31 unreachable, 11 paused, 7 refused. On the first site read (Sial), 25 of 46 books were new to the index. Recheck with every terms page (2026-11-06, `sites.js recheck`): 4 more sites forbid it (prag and Kinneret forbid robots, Shalem Press and Amutat Yam forbid copying) and their 239 books were removed; sentences about returning opened products or spreading viruses are no longer read as copy bans.
- **Web search for misread spines** (`server/src/websearch.js`, step "Web search"): a spine reading no catalogue knows is searched on the web (SearXNG on the server, with the free Brave and Yandex engines behind it; no paid search API). To play nice with those engines the server sends one search at a time, at least 10–15 seconds apart, at most 40 an hour and 300 a day, pauses 10 minutes when the engines answer nothing, and keeps every answer for 30 days so the same query is not sent twice; past the limits a reading simply stays as read. The vision model picks this book's title and author out of the results, and the correction is kept only when it is written in the results as given and at least 3 of every 4 of its title words are close to a word read on the spine (look-alike letters count as close); when the spine showed an author, one of the names must be close to it. The corrected title is then looked up in the catalogues again. If the whole reading finds nothing and the title has parts ("author • title"), its longest part is searched too. Tested on 8 spines: "צ'יטרה" → "אדונית התבלינים" → "ארונית התבלינים", "אתוות החוף" → "אחוזות החוף" (אמיר גוטפרוינד), "יפתח ספקטאור" → "רם וברור" by "בנרג'י דיוואקרוני"; readings the web does not know are left as read.
- **Confidence and order in the approval queue**: each book from a shelf scan gets a confidence from its evidence (`server/src/confidence.js`): barcode found in a catalogue 95; **sure** 92 (found in a catalogue, reading confirmed by OCR, the author read on the spine agrees with the record, the title read is the record's main title, and the book was seen in two

photos or more); found in a catalogue with the reading confirmed by OCR 90; found in a catalogue 85; corrected and then found 75; found, but no author was read on the spine or it is unlike the record's 70, or the spine's cut-out is short and wide (under 4 times as long as high: books lying flat, or two spines) 70; known only to the local index 65; found, but the record names no author 60, or an index correction also replaced the author by an unlike name 60; corrected by the web only 55; found, but the record is by another author 50; found from the whole-shelf photo only 45; OCR-confirmed reading 40; the same, but a web search found nothing 30; read by the vision model alone 25; weak reading 15. The queue shows the surest first, each with 🟢 (75+), 🟡 (45–74) or 🔴 (under 45), the percentage and the reason. Older items get 85 / 40 / 20 from their quality.

- **Calibration** (10 Nov 2026, `docs/CONFIDENCE_REVIEW.md`): on 218 high-confidence decisions the old scores agreed with the reader 84% of the time; the rules above put 172 books at 75 or more with 94% agreement, and 49 books at "sure" with 96%. The reader's reasons for the 9 right books declined anyway were all the picture: flat stacks, two spines in one cut-out, a sliver, one upside down. With the short-and-wide cut-out rule, 164 books score 75 or more with 96% agreement, and all 46 "sure" books were approved. A cut-out is no longer turned upside down unless Tesseract's text really holds the title or author (fit 0.5 or more).
- **Poor cut-outs are not shown**: a spine cut-out that is short and wide (under 4 times as long as high: books lying flat, two spines) or more than a quarter blank is kept for the reader in the queue, with the note "This cut-out is poor and won't be shown on the site. If the book is right, approve it.", but is left out of the library and book pages and not sharpened. Its shape is saved with the item (`scan_items.crop_shape`); `scripts/crop-shapes.js` measured the older ones. `scripts/backtest-confidence.js` scores every decided book again with the current rules and compares (nothing is changed).
- **Auto-approval of sure books** (off unless `AUTO_APPROVE_SURE=1`): books scored "sure" are approved without the reader once their scan is read, after duplicates are merged. They show in "Recent decisions" and can be undone like any other decision.
- **Where each book's reading came from** (table `scan_item_evidence`, view `evidence_by_source`): every book a scan sends to the queue keeps one row per piece of evidence: how it was read (barcode, vision model, vision model confirmed by OCR), each change to the reading (another frame's spelling, the web search, with the query, whether it came from the cache, and the sites and engines whose results spelled the correction), each catalogue that checked it (local Open Library, National Library, Open Library, Google Books; match or not), and the reader's decision (kept as read, edited (with the title kept), or declined). `evidence_by_source` counts, per source and outcome, how many of those books the reader approved as they were, edited or declined: which sources help and which mislead.
- **Catalogue match check**: a book found by title (or by a printed Danacode) is accepted only when it fits the covers: every word of its main title on one line of the cover text, a word of a long subtitle somewhere on the covers, and one of its authors among the names on the cover (Hebrew names are compared with romanized ones by their consonants: "Carman" = "קרמן"). Otherwise the book waits with the title as read. A title found inside a longer catalogue title no longer counts as a match.
- **Which way up**: Surya and the vision model both read upside-down Latin print, so an English or Spanish spine (they run bottom to top, Hebrew ones top to bottom) used to be kept upside down when both ways read the same. When both ways are confirmed, Tesseract, which reads upside-down print badly, now decides; it turned 4 of the 5 upside-down crops of 7 Nov 2026 the right way and left the 2 right ones alone. `scripts/upright-crops.js [--apply]` fixes saved pictures of open items.
- **Books lying flat**: a long level cut-out (2.5 times wider than tall) is cut to the spine and straightened like an upright spine, instead of keeping the cover around it.

- **Cover blurbs are not titles**: “INTERNATIONAL BEST SELLER”, “OVER 10,000,000 COPIES SOLD” and the like are dropped before any lookup. A Latin reading no longer matches a record only through its alternative titles and subtitle (“LEE CHILD” had matched “Bloomsbury Murder: ‘A-grade crime fiction’ Lee Child”); that path is for Hebrew read against romanized records.
- **Only the book’s authors**: organisations (theatres, councils, museums, publishers) are dropped from a record’s author list, and in a Hebrew book whose record names someone in Hebrew, the Latin-letter names next to them (adapters, translators, producers added by Library of Congress style records: “אשה בורחת מבשורה” listed דויד גרוסמן, Hanan Senir, Teatron ha-Kameri shel Tel-Aviv, Habimah). The list is never left empty. `scripts/hebrew-titles.js --apply` cleans saved books.
- **Hebrew author names**: a Hebrew book whose catalogue record names the author in Latin letters gets the National Library’s Hebrew name (a short title such as “הלב” is searched with the author’s surname; “הר אדוני (מונטדידיו)” counts as “הר אדוני”), or else the Hebrew name read off the spine when it is the same person.
- **Hebrew titles stay Hebrew**: Open Library often gives a Hebrew book a romanized subtitle (“מה לאכול: ekh li-veḥor mazon…”, about 16,800 records). When a book is saved, the National Library’s Hebrew title of the same book (by ISBN, or by the main title and the same author) replaces it; failing that, the main title alone is kept. `scripts/hebrew-titles.js [--apply]` fixes books saved before 6 Nov 2026.
- **Title and author read the wrong way round** are swapped before the lookup when the “title” is a known author’s name (85%+ alike in the local index of 280,000 names) and the “author” is not; two names are left alone, and so is a one-word “title” (“היום”, “דילמה” are also names in the index: 7 false swaps up to 8 Nov 2026). On the readings readers had corrected by 6 Nov 2026 it swapped 5 of 6 and none of the 8 right ones. (The name lookup used to keep the first 20 candidates alphabetically instead of the closest, and missed most names.)
- **A book by someone else is never a match**: when an author was read off the spine, a catalogue record naming other authors is rejected, even when the title is the same (names are compared by words and letters, so a misread name still counts; Hebrew with romanized by consonants). Until 6 Nov 2026 a lookup that failed with the author retried the title alone and took any record: 6 of the 33 declines that week were such books (“משוואה עם נעלם” by אורי אדלמן matched a 2005 study by other authors). A record naming no author at all needs nearly the whole title (85%) when an author was read. `server/scripts/rematch.js [--apply]` runs the open queue through the current matching again.
- **Codes**: barcodes, ISBNs printed on the back (check digit verified) and Danacodes (trusted only when the catalogue record passes the check above).
- **Catalogue lookup**: a local copy of Open Library (38.6 million books, refreshed monthly), then the National Library of Israel, Google Books and Open Library online. Covers and descriptions for book pages come from the National Library of Israel and Open Library, fetched under the name SeferehovBot (Simania was dropped on 8 Nov 2026: its terms forbid bots and copying; Amazon covers were dropped the same day).
- **National Library of Israel and Hebrew author names**: the National Library is read through its catalogue’s public SRU interface (Alma, MARC records, no key); its older search API went behind a browser check in November 2026. Names come from the MARC fields marked Hebrew, and only real authors are kept: the main author, and added names marked as authors or named before the “;” of the credit line (translators, editors and illustrators come after it). Hebrew books from Open Library carry library romanization (“Menaḥem Miśgav”); when such a book is saved, the National Library record of the same book gives the names in Hebrew (“מנחם משגב”): found by ISBN, or by the exact title when one of its authors has the same consonants as a romanized name. Failing that, the marks are dropped (“Menahem

Misgav”). `server/scripts/nli-hebrew-names.js` fixed the books already saved: of 60 Hebrew books with romanized names (2026-11-06), 37 now have Hebrew names and the rest plain letters.

Covers: more passes (8 Nov 2026)

Of the first 40 covers, 16 needed editing and 6 were declined. Covers now go through, in order, until a catalogue knows the book: 1. **Reading**: the vision model and Surya OCR; on Hebrew covers the 3 biggest lines (title, author) are cut out, enlarged and read again by Surya and Tesseract, and the readings vote (hspell breaks ties). 2. **Codes**: barcodes and printed ISBN/Danacode; an ISBN no catalogue knows is also looked up among the books publishers list on their own sites. 3. **Corrections like spines**: title/author swap, the local title index, the web check. 4. **Author first**: the author read well and the title badly: the author’s books (local index, National Library, else one web search) held against the cover’s text; the one whose title is printed there is taken. 5. **The blurb**: a sentence of the back cover (or a 7+ word “title”, a back photographed alone) searched word for word; pages quoting it name the book. Every step is recorded as evidence and gives the item a confidence for the queue. Re-reading the 20 covers readers had fixed or declined: 15 came out right with no edit, 2 matched another book by the author read, 3 were not found. Author-first needs the title on one line of the front cover and a record naming the same author (it once matched a memorial booklet whose three common title words were all in a blurb). Searching a Danacode on the web finds nothing and is not used. `scripts/cover-test.js [scan…]` re-reads saved covers without saving.

Web searches: engines in turn, and a queue (8 Nov 2026)

- Each web search asks one engine, in turn: Brave, Yandex, Yahoo (Mwmbl, a small open index, when the others rest). Each engine has its own limit (15 an hour, 100 a day; Mwmbl 10 and 60) and rests 30 minutes when it refuses (a CAPTCHA, “too many requests”, or nothing at all).
- A web check that cannot be searched when a scan is read (every engine resting or used up) is queued (`web_jobs`) and run later, one at a time: every 3 minutes by day, every minute at night (00:00–07:00 Israel time). A result updates the item still waiting in the queue (title, author, the catalogue’s book, confidence) with its evidence; a check still held is tried again half an hour later, up to 6 times. Items decided meanwhile are left alone.
- Searching from readers’ phones was considered and dropped: browsers cannot read search engines’ result pages (no CORS), and spreading searches over phones to get past engines’ limits would go against their terms.

Printed books only, and whole-shelf readings marked unsure (8 Nov 2026)

- A catalogue record of an audiobook or an ebook (publisher “… Audio”, Audible, Blackstone; “(Unabridged)”, “Kindle edition”, EPUB, “ספר דיגיטלי”, “ספר קולי”) is never matched to a book on a shelf. “The Millionaire Next Door” had been matched to the Simon & Schuster Audio record.
- A book read only from the whole-shelf photo, with no cut-out of its own spine, gets confidence 45 (“found in a catalogue, but read only from the whole shelf”) even when a catalogue knows it, so it is shown among the unsure ones. Such matches (“The Millionaire Next Door”, “3rd Degree”) were declined.

The approval queue

- Every book found, from any photo or barcode, waits in one queue per reader (`/queue`), one book at a time with a big picture: approve, later, or wrong. The title and author can be fixed before approving.
- Poor shelf reads stay out of the queue: no catalogue knows them, and the read is low quality, title and author are the same words, no word of a Hebrew title is a word, or it is a misread of a book the catalogue found in the same scan. Nothing is deleted: a button at the end of the queue brings them back. Covers are never left out.
- A spine cut badly in every photo it is in (25% or more of the cut-out is blank fill: the cut ran off the spine) and known to no catalogue is given up. On 6 Nov 2026 every such crop had been declined and no approved crop was over 20% blank. When a spine is in several photos, the cut-out with the least blank (then OCR-confirmed) is the one shown.
- Duplicates from several photos are merged, also when title and author are compared word by word in any order with a misread letter or two ("ברק חמדני אבק" = "אבק | ברק חמדני אבבק": one spine found twice in a photo). Decisions can be undone for 14 days.
- "How this book was read — step by step" (a fold-out under each book, `web/src/steps.js`, `GET /api/queue/:id/steps`): the steps of the scan's trace that made this one book, each with its picture — the photo as it reached the server, the boxes the model drew (this book in red, the lean measured in it), the photo turned for leaning books, the cut-out with its size in original pixels, each way up the vision model and Surya OCR read it and how far they agree, the final level cut shown for approval, a spine found by its title, the whole-photo read, and the catalogue lookup with its verdict. Only the reader who scanned it, or an admin. Each book records where in the trace it was read (`scan_items.trace_at`); older books are matched by title.
- The page tells readers that each correction or "wrong" helps improve reading for everyone. Those scans (photos without location data, trace, and the decision) are kept for good to study reading errors.
- A reader who closed the app gets an e-mail when the results are ready.
- While photos are still being read, the queue page says how many and about how long until they are done.

Reading order (when many readers scan at once)

- Book covers go first: a cover sent while a long shelf batch is being read is read between two of its photos, within a few minutes.
- Readers take turns: everyone's first shelf scan before anyone's second, so one big home library does not hold up everyone else.
- The expected wait comes from the last 30 days of measured reading times.
- **A long queue** (`server/src/busy.js`, 10 Nov 2026): every 10 minutes the server works out how long shelf photos sent now would wait. When that passes 3 hours (`BUSY_ALERT_HOURS`), the admins get one email; the next comes only after the wait has fallen under half that. On `/admin#queue` the admin sees who waits an hour or more and the email they would get, and can send it with one click: "Many people sent shelf photos today, so reading them takes longer than usual. Yours should be ready in about N hours. We will email you when they are." (in the reader's language). Each waiting scan is told at most once; nobody is emailed about it without the admin's click.

My books (home library)

- Books scanned at home stay private. Shelf photos or covers taken from My books become the reader's own books.
- **"I left it in a library"**: one tap reports the book as added to that library.
- Books of friends, or borrowed from friends, can be kept apart from the reader's own.
- With more than 5 books, the list can be filtered by words, genre and language, and sorted by when added, title, author or year.

Where is it? (from 8 Nov 2026)

- Every spine read off a shelf photo keeps its outline on that photo (`scan_items.spot`: four corners as fractions of the photo, worked back from the straightened cut-out). The queue's photo is the one the outline is on.
- A book's page shows **Where it is**: each shelf photo it was seen in, darkened except for its spine (outlined in yellow), the books to its left and right in the photo (or above and below, for a pile of books lying flat), and when.
- Your own home scans: shown only to you. You can give a scan a place name ("living room, left bookcase"); every book from that photo shows it. A street library's photos: shown to anyone once the books were approved, with the library's name.
- Spines scanned before 8 Nov 2026 were located afterwards by the vision model, by title (`scripts/spots.js`; a box, not the exact lean).

Books for sale or to give away

- Any book in My books can be offered: free or for sale (price in ₪), its condition, the town and a note.
- Only the offered book is public, with the town and the reader's screen name. The address, e-mail and the rest of the home library are never shown. Phone numbers and e-mails in the note are refused.
- A buyer writes through the app. The message is e-mailed to the owner, with the buyer's address to reply to (the buyer is told before sending). Limits: 10 messages an hour, 20 a day.
- Offers appear on the public book page (with schema.org `Offer` data, so search engines can show price and condition) and on `/offers`, filtered by town and by free or for sale.

Finding books

- **Search** by title or author: within 1, 5, 20 or 50 km of the reader or of a town, or anywhere. With no words it lists all books in the area ("Browse all books" lists every book on a shelf).
- **Filters and sorting** on the results: genre, language, period published (before 1950, 1950–79, 1980–99, 2000s, 2010s, 2020 on) and when last scanned (week, month, year). Each choice shows how many of the books found it would leave. Sort by best match, scanned lately, nearest, most copies, newest or oldest, or title. Each book shows its genre, language and year, on how many shelves it is now, how many times readers scanned it, and when it was last seen.
- **Genres** (23, from novel and poetry to Judaism, cooking and reference) are seldom given by catalogues, and never for most Hebrew books, so the local model picks one from the title, authors, publisher and description, in the background when the GPU is idle (about a second a book). A missing language is taken from the script of the title, or from common English words,

else guessed by the model. Readers can correct both in “Fix a mistake”, and the book page links them to the matching search.

- **Alerts** (watches) for a book, an author or words, near a place or anywhere, by e-mail or push notification.
- **Public book pages** (`/b/<id>`): covers, cover text, ISBN and Danacode, where the book is now, offers, reviews, and links to Open Library, Google Books, WorldCat and Simania.
    - **Titles in the words people search with** (12 Nov 2026): “ארוחת הערב / הרמן קוך — בספריית רחוב בחיפה | ספרחוב” (“… by … — free at a street library in …” in English; “given away free in …” or “for sale second-hand in …” when offered). The description starts the same way, then the blurb. (Was “title — author | Seferehov”, and the blurb alone.)
    - **Wrong books kept out of search engines** (`titleFits` in `server/src/seo.js`): the page’s title must be found in what was read off the book’s own photos (its approved spine readings and cover text). Any part of the title will do (the main title, or a subtitle of two words or more), a word may lack its first or last letter (Hebrew prefixes, OCR slips), י and ו count as one letter, and half the main title will do if an author’s name was read too. A book with nothing read (a barcode, typed by a reader) is trusted. A page that fails gets `noindex` and is left out of the sitemap; `scripts/book-fit.js` lists them for a person to fix. First run: 4 of 289 pages, 3 of them really another book (book 95, a cover reading “ארוחת הערב / הרמן קוך”, was a 1988 dinner booklet) and one right book read in Russian.
- **Reader pages** (`/r/<id>`): what a reader read and recommends. Home libraries are not shown.

Reviews, fixes, points

- **Reviews**: “I read it”, stars, and four guided questions. Copied text and contact details are refused; admins can hide a review.
- **Book fixes**: anyone can fix a book’s title, author, description and so on. Trusted readers’ fixes go live at once; others are voted on. Every change has a history and can be undone.
- **Points and levels** for approved reports, with a daily cap. New readers’ reports are reviewed first; after a few good ones they publish directly. Reports made far from the library always go to review.

App and updates

- **The menu’s to-do count shows at once, without polling**: the counts are kept on the phone (with the reader’s id; cleared on sign-out) and the menu opens with them. They are asked for again only when the app starts, when it comes back into view after 10 minutes or more, or when the menu or a to-do list opens and they are over 2 minutes old; the replies to the reader’s own approvals, undos and uploads update them. The reader’s queue count is one shared copy for every screen (it used to be fetched on every screen change, and by several screens again).

- **Load at scale** (review of 10 Nov 2026, thinking of 1,000 readers at once):

    - Work about all the data runs in a **daily server job** (`server/src/jobs.js`), not in requests: duplicate book candidates (every book against every other), duplicate libraries, and per library the books needing cover photos and the spines waiting for covers (tables `dupe_candidates`, `library_needs`). The to-do count and the duplicates pages only read them; a merge or “not the same” removes its pair at once. New duplicates show up the next day.

    - The numbers that are the same for every trusted reader (reports to review, duplicates, cover texts) are read once a minute, shared. Statistics, the leaderboard and the “where you can help” list are kept 5–10 minutes; scan waiting times 5 s. Concurrent requests share one computation in flight.
    - The approval list no longer changes anything when it is read: misreads are set aside and duplicates merged once, when a scan is finished.
    - Screens waiting for the server (the queue while photos are read, the shelf check, a book’s details) look again after 8 s, then longer up to a minute, not while the app is out of view; a book’s details at most 8 times.
    - The map asks for areas snapped to a grid of half map tiles (small moves ask for nothing new), and the approved libraries in an area are kept a minute for everyone; a reader’s own libraries waiting for approval are added on top.
    - The connection check and upload retries have a random spread (no phones in step after an outage); a signed-out phone’s waiting uploads wait longer each time instead of every minute; the new-version check runs once per version seen.
    - Indexes on the columns hot requests filter on (libraries by who added them, sightings by reader, scans by library, scan items by book / sighting / my book, my books by book).
    - Not changed: the app’s offline copy still includes the barcode reader (1 MB), so the camera works at a library with no signal.

- **Polite to the catalogues** (`server/src/polite.js`, 10 Nov 2026): every request to the National Library, Open Library and Google Books, from scans and from book pages, goes through one gate per site. Requests to a site start at least 1 s apart (the National Library 1.5 s); at most 30 wait their turn and the rest are refused at once; a site that answers “too many requests” (429) or “unavailable” (503) is left alone for the time it asks, or 1, 2, 4… up to 30 minutes. A book page whose catalogues were too busy is looked up again on a later visit instead of being saved as “nothing found”.

- Installable from the browser (PWA), works offline, Hebrew and English.

- Server not answering (no signal, a stale DNS address, the server down): screens never wait on a blank page. A screen shows “Loading…” until it draws; after 4 s a bar under the header says the server is slow; screens seen before open after 3.5 s from the last answer the phone kept (marked “showing what the phone saved last”); who is signed in is kept on the phone, so the menu and screens don’t flip to “signed out”. The bar asks the server again every few seconds (and on “Try again”) and goes away on the first real answer. Sign-out deletes the kept answers.

- **Comic** at `/comic` (“How it works” in the ☰ menu, linked from About): nine drawn panels explaining why Seferehov exists (gems on street shelves are found only by chance) and how it works (photograph a shelf, approve the books, search by title and distance, alerts, take and leave books). Bubbles and captions are page text in both languages (each caption a `<figcaption>` of its picture, every picture with a Hebrew or English description), also rendered by the server for search engines: ComicStory structured data listing the nine pictures with their descriptions, a 1200×630 share picture with its description, and the pictures listed in the sitemap. Newcomers see a one-line pointer to it on the map until they open it or close the line. The drawings were made with FLUX schnell on the home server; they load only when the comic is opened.

- **English page for developers** at `/en` (13 Nov 2026, `server/src/enpage.js`): how Seferehov reads shelves with local AI on one GTX 1080 — the hardware, the pipeline step by step, the measured accuracy table and how it was built with Claude Code — with links to the app in

English, the comic, the write-ups on amiheines.com and the Local AI Pilot. A plain server page, outside the app (the service worker leaves `/en` alone); pictures in `web/public/en/` (versioned names); listed in the sitemap.

- Updates install in the background and switch in between screens, never during an upload or on the camera, queue or review screens. No version numbers to bump.

For the admin

- `/admin`: activity, readers, search pages, cover texts waiting for approval, reviews, statistics.
- `/admin#queue`: how long new shelf photos would wait, who waits an hour or more, and the "longer than usual" email to send them (see *Reading order*).
- `/admin#camera`: every live-camera session (shelf, covers, many books, barcode; also ones that sent no photo or whose camera would not open): phone (model · system · browser), camera resolution, time to open, photos and how each was taken, median time from ready to photo, books and front→back time, barcodes found, share of time holding, moving on, too fast, dark and glare, median speed, how it ended, and the full numbers; below, the last 100 uploads with their times on the phone, upload and server.
- `/admin/trace`: every AI stage of every scan, step by step: the video and the frames chosen, each cut-out spine, what each model was asked and answered, and why each book was kept or dropped. Test uploads run the full pipeline without saving anything.
- `/admin/outlines` (14 Nov 2026, for a laptop): spine outlines by hand. Two lists: approved books with no outline on their shelf photo, and outlines on one photo that lie on top of each other. Opening a book shows its photo full-window with every other outline on it (overlapping ones in orange) and the cut-out the app read: drag to draw a box (it may go past the photo's edge, for a spine the photo cuts), drag a corner to fit a leaning spine, drag inside to move it, `[ ]` to turn it, the wheel to zoom; *Save and next* walks through the lists. *Not visible* marks a book that can't be seen on its photo (`spot = {hidden: true}`, so `scripts/spots.js` doesn't look for it again). Hand-made outlines are `{poly, by: 'admin', at}`; they show on the book page and become training data for the lab's spine finders. **Suggestions:** the lab's spine finder (YOLO, run at night) proposes outlines for books without one, where no other book is outlined (`OUTLINE_SUGGESTIONS`, a JSON file the lab writes); they show dashed green, a click takes one, and when there is a single one (or the lab marks a best guess) it is already in place, so approving is one key (Enter).
- **Photos and training** (About page, 14 Nov 2026): shelf photos of public street libraries may be used to train and test the spine finder; photos of home libraries only with the photographer's permission (the lab trains on scans with a library and no owner, plus the admin's own home scans, by his permission); models may be published, the photos never as training data.
- `server/scripts/eval.js`: a fixed test set for measuring reading accuracy (see below).
- Search statistics: which catalogue answered each search and whether the reader kept the match.

Measured accuracy (2026-10-21)

On 60 synthetic Hebrew back covers with known text, word accuracy was:

Reader	Clean	Like a phone photo	Hard (tilted, blurred, noisy)
Surya OCR	98%	97%	98%
Surya + hspell	97%	96%	97%
Vision model	92%	76%	77%
Tesseract	72%	73%	28%

On real photos, the title was found in:

- Hebrew spines: 71% of spines by the vision model alone, 63% by the full pipeline;
- Hebrew covers: 63% by the vision model, 49% by Surya;
- Latin covers: 93–100%.

Real Hebrew covers are harder than synthetic ones, so a word-accuracy figure for them needs a hand-labelled set.

Tests (from 11 Nov 2026)

`npm test` runs four checks, at three points:

- **Before every push** (`.githooks/pre-push`; turn on once per clone with `npm run hooks`): each commit being pushed is tested in a clean copy of it, so uncommitted changes don't count. A failure stops the push and prints the failing lines. `git push --no-verify` skips it in an emergency.
- **On GitHub** after every push (`.github/workflows/test.yml`): the same checks again, on GitHub's machines. Free for a public repository. A failed run sends an email.
- **On deploy** (`deploy/deploy.sh`): a commit that already passed before its push (noted in `.git/tested-commits`) is only built; otherwise the tests run first and a failure stops the deploy. `SKIP_TESTS=1` skips them.

Branch protection on GitHub ("master only accepts commits that passed") is not used: it works on pull requests, so every change would have to go through a pull request instead of a push. The check before the push gives the same guarantee for a single developer.

1. **Lint** (`npm run lint`, `eslint.config.js`): names used but never defined, unused code, and similar mistakes, in the phone app and the server. It caught a "No, still fronts" button in the covers camera that had stopped working.
2. **Unit tests** (`npm run test:unit`, `server/test/`): the confidence rules, the catalogue gate, the shared cache.
3. **The build** of the phone app.
4. **End to end** (`npm run test:e2e`, `test/e2e.test.js`, about 1.5 minutes): a fresh database with every migration and a little test data, the real server, and headless Chromium. It opens every screen as a visitor, a reader and an admin, in Hebrew and English, and opens the menu on each. A screen fails if the page throws an error, logs one, gets a server error from the API, or still shows "Loading…" after 6 seconds. It also calls every GET endpoint of the API (any server error fails) and checks the server logged no errors. Nothing outside is asked: no catalogues, vision model or mail. It needs Docker for the database locally (CI uses its own) and Chromium (`npx playwright install chromium`).

Checked on the to-do tabs bug of 11 Nov 2026 (screens stuck on "טוען"): lint reports `'html' is not defined`, and the end-to-end run fails on /queue and /review.

Hardware

Everything runs on one home server:

Part	
CPU	48 threads (AMD Threadripper)
Memory	251 GB
GPU	NVIDIA GeForce GTX 1080, 8 GB (from 2016)
Disks	1.8 TB NVMe (1.5 TB free) for the app; a separate hard disk for the nightly backup
Software	Node 22 + Fastify API, PostgreSQL 16 + PostGIS (Docker), nginx, Ollama (vision model on the GPU), Surya OCR (on the CPU), hspell

The same server also runs other services. The GPU is shared with other occasional jobs through gpu-queue (since 8 Nov 2026): every program that uses the card waits its turn there, and the card is emptied before it passes from one program to another. Seferehov's photo reading and cover sharpening run at normal priority: a page where someone is waiting goes first, and bulk background work goes after. A photo may wait a few minutes while another program uses the card, so the reading time-out is 10 minutes (`VISION_TIMEOUT_MS`).

Capacity: how many scans the GPU can handle

The GPU reads one scan at a time; others wait in a queue (covers first, then readers in turn; see *Reading order*). Measured times (the last five shelf scans and 40 cover scans, November 2026):

Scan	Time
One book's front and back covers	about 1 minute
One shelf photo	about 9 minutes (7–14): about 45 vision-model calls of about 9 s, and Surya OCR on the CPU taking about as long
A visit to a library (about 8 shelf photos, about 8 books each)	about 75 minutes

So in 24 hours of non-stop work, with no other program using the card, the server reads about **160 shelf photos (about 1,250 books), or about 20 library visits**. More than that every day and the queue grows without end.

Load	What happens
Up to about 20 visits a day (for example 140 readers each scanning once a week)	Results within about an hour
40 visits a day, every day	The backlog grows by 20 visits a day: about 18 hours' wait on day 1, a week by day 7
An evening peak: 15 visits from 17:00 to 20:00, quiet otherwise	About 18 hours of reading; results by the next day, and the queue empties overnight
A burst (an article, a TV mention): 100 readers × 8 photos in 3 hours	About 5 days to clear
300 readers × 8 photos	About 15 days; readers take turns, so everyone's first photo is read within about 45 hours

Nothing breaks under load: photos wait on the phone and on the server, the queue shows the wait, readers get an email when theirs are read, and the admin is told when the queue is long (see *Reading order*). The harm is readers waiting days and losing interest. The second limit is the web search: about 360 searches a day across the free engines, used up at full load, so unknown books get their search days later.

The bottleneck is the GPU, not the CPU, memory or disk.

Ways to grow, in order

Open issues (10 Nov 2026): #3 parallel OCR, overlapped with the vision model (about 1.5–2×); #4 a quick pass when the queue is long; #5 fewer vision-model calls per spine; #6 a used RTX 3090 (estimated 3–4×). #3 and #6 together: about 100 library visits a day.

1. **Smarter queue (code only, free). Done.** Covers before shelf batches, readers in turn, and an estimated wait shown in the app. This removes the worst waits but not the total: five home libraries at once still take 10+ hours in all.
2. **Faster pipeline (code only).** Each spine is read four times now (two orientations, two readers); reading the likely orientation first and skipping the second read when both agree would cut the time considerably.
3. **A better GPU at home.** A used RTX 3090 (24 GB) costs about $700–900 and is roughly 3 times faster for this kind of model (its memory is about 3 times faster). The power supply has to be checked (the card draws up to 350 W). This keeps the "no per-use cost" property.
4. **A rented GPU for overflow.** The worker takes jobs from the database, and the model address is a setting, so a second worker on a rented GPU server can take jobs only when the queue is long. Approximate on-demand prices (October 2026, check before buying):

Provider	GPU	Per hour	24/7 for a month
AWS EC2 g4dn.xlarge	T4, 16 GB (about as fast as the GTX 1080)	$0.53	$384
AWS EC2 g5.xlarge	A10G, 24 GB	$1.01	$734
OVHcloud	L4, 24 GB	about $1.00	about $730
RunPod community cloud	RTX 3090 / 4090	$0.19–0.39	$140–280

Run only when needed (for example 2 hours a day), a rented GPU costs tens of dollars a month. Running one all the time costs hundreds, more than buying a faster card. A rented server is still a

machine the operator controls: photos go to it, not to an AI company, and they are not used to train anyone's model.

Disk space

Measured per item:

Kept	Size	How long
Book cover photos (front + back, re-encoded)	about 0.6 MB per book	permanent
Shelf photo for the queue + spine cut-outs	about 0.3 MB per photo + 40 KB per book	permanent
Full-size copy of each photo as sent and as read, and the video as sent, without location data or sound (for reading again with a better model)	about 3 MB per photo, 20–100 MB per video	1 year
Scans where a reader declined or corrected a reading: those copies, the trace and what the reader decided (to find where reading goes wrong)	as above	permanent
Trace of each scan (admin view)	0.3 MB per cover, 1.6 MB per shelf photo, 25 MB per video	1 year
Database: Open Library catalogue	14 GB	rebuilt monthly from the public dump
Database: readers' data	a few MB so far	permanent

The 1-year files are capped by the GPU: at most about 400 photos a day can be read, so they stay under about 700 GB, and far less at real use. Permanent growth is small. For example, 1,000 home libraries of 300 books photographed by shelf come to about 20 GB, and 100,000 books scanned by their covers to about 60 GB. With 1.5 TB free, disk space is not a concern for years.

Backups

Now: every night at 04:00 the server copies the app files to a second disk in the same machine and dumps the database (the last 7 dumps are kept). Files that were replaced or deleted are kept for 60 days. A failed backup sends an e-mail.

This protects against a failed disk, but not against anything that hits the whole machine or house: theft, fire, a power surge, or a mistake or attack that reaches both disks.

Recommended:

- **Dump the readers' data without the Open Library catalogue.** The catalogue can be downloaded again, and without it the nightly dump shrinks from gigabytes to megabytes.
- **A second machine on the home network: a Raspberry Pi with a USB disk works well.** A Pi 4 or 5 with a 1–2 TB USB SSD or hard disk is enough. Use the wired network if possible;

Wi-Fi is fine too, because the nightly change is small (megabytes).
    - Let the Pi **pull** the backup over SSH with a read-only key, so a compromised server cannot delete the backups.
    - Keep dated snapshots (for example with borg or restic), not a single mirror.
- **One copy outside the house**, encrypted (borg or restic to a rented server or cloud storage). The important data is under a few GB, so this costs cents to a few dollars a month.
- **Test a restore** once in a while.

Future plans

Waiting and speed

- Faster shelf reading (read the likely orientation first; skip the second reader when the first is sure).
- Overflow worker on a rented GPU when the queue is long; or a faster GPU at home.

Home libraries and offers

- Alerts to readers watching a book when someone offers it.
- Offered books in search results.
- A “wanted” list: readers post books they are looking for, matched against offers and (with the owner’s consent) home libraries.
- Marking an offered book as sold or given, with a thank-you to the other reader.

Second-hand bookshops

- Shop mode: photograph the shelves to get an inventory that can be searched and priced, with no typing.
- Photograph again later: sold books drop off the list.
- Export of the stock to the shop’s own website or to classified-ad sites; a page for shops with measured numbers from a pilot.

Accuracy

- “Already known”: while a cover or spine is in view, the phone sends a small picture fingerprint (the image embedder it already has); if the book is known with both covers, it says so and the back is not needed. Spines seen before recognised without reading them.
- Use the phone’s motion (logged with every free-style photo) in the pairing, once there is data on what turning a book over and moving on look like.
- Harder synthetic test covers (large decorative titles, text over pictures) and a small hand-labelled set, so accuracy on real covers can be measured and published.
- A public “try it” page that shows each AI stage on an uploaded image.

Operations

- Off-machine and off-site backups (above).

Keeping this document current

This file (`docs/FEATURES.md`) is updated with every feature change. The PDF is built from it with `docs/build-pdf.sh`.